

V1 Specification: E-Commerce Storefront with Wishlist Merge

Author: Senior Staff Product Engineer
Date: 2026-07-15
Status: Draft — Pre-implementation
Version: 1.0.0

Table of Contents

- 1. [Requirement Analysis](#)
 - 2. [Ambiguities](#)
 - 3. [Assumptions & Rationale](#)
 - 4. [Product Scope \(90-Minute Build\)](#)
 - 5. [Functional Requirements](#)
 - 6. [Non-Functional Requirements](#)
 - 7. [Data Models](#)
 - 8. [Merge Behavior](#)
 - 9. [Edge Cases](#)
 - 10. [Tradeoffs](#)
 - 11. [Out of Scope](#)
-

1. Requirement Analysis

Original requirement:
"Build a simple e-commerce storefront with a wishlist feature. The only mandatory requirement is that users must be able to merge two independent wishlists into one."

This is intentionally under-specified. It gives us a domain (e-commerce), a feature category (wishlist), and a single hard constraint (merge two wishlists). Everything else must be reasoned from first principles.

2. Ambiguities

The following are the explicit ambiguities that must be resolved before any line of code is written:

#	Ambiguity	Why It Matters
A1	What does "simple" mean?	Determines feature scope, number of pages, polish level
A2	What does "independent" mean in "two independent wishlists"?	This is the core architectural question — same user's named lists vs. different users' lists vs. guest + auth
A3	What is the source of truth for a wishlist after merge?	Determines whether merge is copy-and-delete, union-in-place, or something else

#	Ambiguity	Why It Matters
A4	Is authentication required to have a wishlist?	Guest vs. authenticated UX is a fundamentally different architecture
A5	What happens to duplicate products during merge?	Must define idempotency behavior explicitly
A6	Is the merge destructive to the source wishlist?	UX and DB write strategy differ significantly
A7	What products exist? Who manages them?	Static seed vs. admin CRUD is a large scope difference
A8	Is checkout / cart in scope?	"Storefront" could imply full purchase flow
A9	Are wishlists named / described / shareable?	Affects data model and UX surface area
A10	Single currency? Tax? Inventory tracking?	Affects Product model and checkout viability

3. Assumptions & Rationale

Each assumption is an explicit decision. If any assumption is wrong, the corresponding section must be revisited before implementation begins.

A1 → "Simple" means: product list + product detail + auth + wishlist management

Rationale: "Simple" in a 90-minute build context must exclude cart, checkout, payment, order history, and admin. The minimum viable storefront that allows meaningful wishlist usage is: browse products, view a product, save it to a wishlist.

A2 → "Two independent wishlists" means: two named wishlists owned by the same authenticated user

Rationale: This is the most common pattern (e.g., Amazon "Create a list", Netflix "My List" variants). If it meant two *different users'* wishlists, the surface area explodes into social sharing, permission models, and conflict resolution. If it meant guest vs. auth merge (anonymous session → login), that's a valid interpretation but adds session management complexity that is disproportionate to the stated scope. Named lists per user is the MVP-correct interpretation.

A3 → Source wishlist is deleted after merge; target wishlist survives with the union of items

Rationale: A non-destructive copy would be confusing UX ("I merged them but now I have three lists"). The user's mental model of "merge" implies one result. Source deletion is the intuitive behavior.

A4 → Authentication (email + password) is required to persist a wishlist

Rationale: Wishlists are inherently persistent and personal. Storing them in localStorage is fragile across devices and tabs. Any realistic wishlist feature ties identity to data. Guest wishlist is explicitly excluded from scope (see §11).

A5 → Duplicate products during merge are deduplicated by product_id

Rationale: If both wishlists contain the same product, adding it twice provides no UX value. The expected semantic of "merge" in this context is set-union, not multiset-union.

A6 → Merge operation is atomic; either both the item copy and the source deletion succeed, or neither does

Rationale: A partial merge (items copied but source not deleted, or source deleted but items not copied) leaves the user in an inconsistent state. This must be a single database transaction.

A7 → Product catalog is seeded with static mock data; no admin UI is needed

Rationale: Building an admin product management UI would consume >50% of the 90-minute budget. The product catalog is infrastructure for testing the wishlist feature.

A8 → Cart and checkout are explicitly out of scope

Rationale: The only mandatory feature is wishlist merge. A purchase flow is a separate vertical that could double implementation time.

A9 → Wishlists have user-defined names and no visibility/sharing settings (private by default)

Rationale: Named lists are required to distinguish them in the merge UI ("Merge 'Birthday Ideas' into 'Home Decor'"). Sharing adds auth surface area that is out of scope.

A10 → Single currency (USD), no tax, no inventory tracking

Rationale: Multi-currency and tax calculation are solved problems that require external APIs. Inventory tracking requires real-time stock writes. Neither is relevant to demonstrating wishlist merge.

4. Product Scope (90-Minute Build)

In Scope

Page / Feature	Description
Product Listing Page	Grid of products, filterable by category
Product Detail Page	Name, description, price, image; "Add to Wishlist" button with list picker
Auth (Sign Up / Sign In)	Email + password, Supabase Auth
Wishlist Management Page	List all wishlists; create, rename, delete a wishlist

Page / Feature	Description
Wishlist Detail View	View items in a single wishlist; remove individual items
Merge Wishlists	Select source + target wishlist, confirm, execute atomic merge

Priority Order

1. Auth (blocks everything)
 2. Product listing (entry point)
 3. Wishlist CRUD (prerequisite for merge)
 4. Add product to wishlist
 5. Merge wishlists ← **mandatory**
 6. Product detail page (nice-to-have polish)
-

5. Functional Requirements

5.1 Authentication

- FR-AUTH-01: User can register with email + password.
- FR-AUTH-02: User can sign in with email + password.
- FR-AUTH-03: User can sign out.
- FR-AUTH-04: Unauthenticated users can browse products but cannot interact with wishlists.

5.2 Product Catalog

- FR-PROD-01: The system displays a paginated/scrollable list of products.
- FR-PROD-02: Each product card shows: image, name, price, category.
- FR-PROD-03: Products can be filtered by category.
- FR-PROD-04: Clicking a product opens a detail view.
- FR-PROD-05: Product detail shows full description and an "Add to Wishlist" action.

5.3 Wishlist Management

- FR-WL-01: An authenticated user can create a named wishlist (name required, max 80 chars).
- FR-WL-02: A user can rename an existing wishlist.
- FR-WL-03: A user can delete a wishlist. Items in the deleted wishlist are also deleted.
- FR-WL-04: A user can view all their wishlists and the item count for each.
- FR-WL-05: A user can view the full contents of a single wishlist.
- FR-WL-06: A user can remove an individual product from a wishlist.
- FR-WL-07: A user can add a product to any of their existing wishlists, or create a new one during the add flow.
- FR-WL-08: A user cannot add the same product to the same wishlist twice (UI prevents this; DB enforces it via unique constraint).

5.4 Wishlist Merge (Mandatory Feature)

- FR-MERGE-01: A user can initiate a merge from the Wishlist Management page.
- FR-MERGE-02: The user selects a **source** wishlist and a **target** wishlist.
- FR-MERGE-03: Source and target must be different wishlists owned by the same user.
- FR-MERGE-04: Upon confirmation, all products in the source wishlist are added to the target wishlist. Products already in the target (by `product_id`) are skipped (deduplication).
- FR-MERGE-05: The source wishlist is deleted after all items have been successfully transferred.

- FR-MERGE-06: The entire operation is atomic. If any step fails, the database is rolled back and the user is notified.
- FR-MERGE-07: After a successful merge, the user is shown the target wishlist with its updated contents.
- FR-MERGE-08: The merge UI shows a preview: item count of source, item count of target, and estimated item count after merge (accounting for duplicates is a best-effort estimate; exact dedup count is shown post-merge).

6. Non-Functional Requirements

ID	Requirement	Target
NFR-01	Performance	Product listing page renders within 2 seconds on a standard connection
NFR-02	Atomicity	Merge operation executes in a single database transaction; no partial states
NFR-03	Authorization	Users can only read, modify, and merge their own wishlists (enforced by Row Level Security)
NFR-04	Responsiveness	All pages are usable on viewport widths from 375px (mobile) to 1440px (desktop)
NFR-05	Consistency	Wishlist item counts are always accurate; no stale cache across pages
NFR-06	Error visibility	Any API or DB error surfaces a human-readable message in the UI (not a raw error code)
NFR-07	Data integrity	wishlist_items(wishlist_id, product_id) is unique at the DB level; application-layer dedup is a defense-in-depth measure only
NFR-08	Security	No sensitive data (passwords, tokens) logged; all auth handled by Supabase Auth

7. Data Models

7.1 products table

```
CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT             NOT NULL,
  description TEXT             NOT NULL DEFAULT '',
  price       NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  image_url   TEXT,
  category    TEXT             NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Notes:

- category is a free-form text field in V1 (no FK to a categories table). Can be normalized in V2.
- image_url nullable; UI falls back to a placeholder.
- No stock_quantity in V1 (see §11).

7.2 wishlists table

```
CREATE TABLE wishlists (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL REFERENCES auth.users(id) ON DELETE  
CASCADE,  
  name        TEXT          NOT NULL CHECK (char_length(name) BETWEEN 1 AND 80),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
-- Index for fast per-user lookup  
CREATE INDEX wishlists_user_id_idx ON wishlists(user_id);
```

RLS Policy:

```
ALTER TABLE wishlists ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "users_own_wishlists" ON wishlists  
  FOR ALL USING (auth.uid() = user_id);
```

7.3 wishlist_items table

```
CREATE TABLE wishlist_items (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  wishlist_id UUID          NOT NULL REFERENCES wishlists(id) ON DELETE  
CASCADE,  
  product_id  UUID          NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  added_at    TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
  UNIQUE (wishlist_id, product_id) -- enforces no duplicate products per list  
);
```

```
CREATE INDEX wishlist_items_wishlist_id_idx ON wishlist_items(wishlist_id);
```

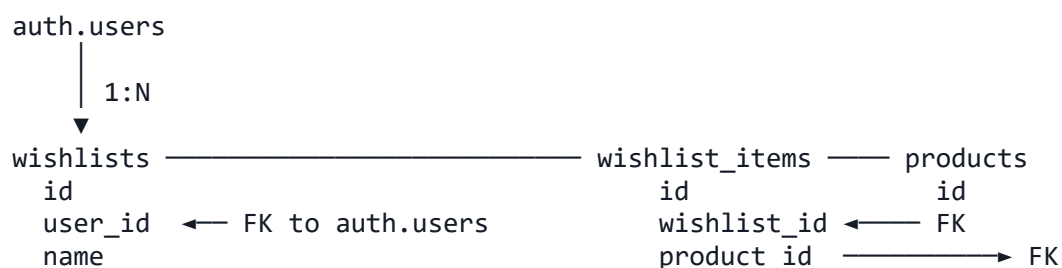
RLS Policy (via wishlist ownership):

```
ALTER TABLE wishlist_items ENABLE ROW LEVEL SECURITY;
```

```
-- Allow access only if the parent wishlist belongs to the current user
```

```
CREATE POLICY "items_via_wishlist_ownership" ON wishlist_items  
  FOR ALL USING (  
    EXISTS (  
      SELECT 1 FROM wishlists  
      WHERE wishlists.id = wishlist_items.wishlist_id  
        AND wishlists.user_id = auth.uid()  
    )  
  );
```

7.4 Entity Relationship Diagram (textual)



created_at
updated_at

added_at

UNIQUE(wishlist_id, product_id)

8. Merge Behavior

8.1 Algorithm

FUNCTION merge_wishlists(source_id, target_id, requesting_user_id):

PRECONDITIONS:

1. source_id ≠ target_id
2. wishlists[source_id].user_id = requesting_user_id
3. wishlists[target_id].user_id = requesting_user_id
4. Both wishlists exist

BEGIN TRANSACTION:

```
1. SELECT all product_ids FROM wishlist_items WHERE wishlist_id =
source_id
2. For each source_product_id:
    INSERT INTO wishlist_items (wishlist_id, product_id)
    VALUES (target_id, source_product_id)
    ON CONFLICT (wishlist_id, product_id) DO NOTHING ← deduplication
3. DELETE FROM wishlists WHERE id = source_id
   -- ON DELETE CASCADE removes all wishlist_items for source
   automatically
4. UPDATE wishlists SET updated_at = now() WHERE id = target_id
COMMIT TRANSACTION
```

POSTCONDITION:

- Source wishlist no longer exists
- Target wishlist contains union of original target items + source items (no duplicates)
- Item count of target ≤ (original target count + original source count)

8.2 Implementation Note

This is best implemented as a **Supabase Edge Function** (or a PostgreSQL stored procedure / RPC) so that the transaction boundary is enforced server-side and cannot be split across multiple client round-trips.

Calling signature:

POST /functions/v1/merge-wishlists

Body: { source_id: UUID, target_id: UUID }

Auth: Bearer <user JWT>

9. Edge Cases

#	Edge Case	Expected Behavior
EC-01	User attempts to merge a wishlist with itself	Rejected client-side before API call; source and target selectors exclude each other's selected value
EC-02	Source wishlist is empty	Merge proceeds; source is deleted; target is unchanged; success message shown
EC-03	Target wishlist is empty	Merge proceeds; target receives all source items; source is deleted
EC-04	All source items already exist in target (100% duplicates)	All inserts are no-ops (<code>ON CONFLICT DO NOTHING</code>); source is deleted; target item count unchanged; user informed that 0 new items were added
EC-05	User has only one wishlist	"Merge" button is disabled / not shown; tooltip explains at least 2 lists are needed
EC-06	User deletes source or target wishlist concurrently in another tab	Server-side precondition check (both wishlists exist and belong to user) fails; transaction is rolled back; user receives a 404-style error with explanation
EC-07	A product referenced in source wishlist was deleted from the catalog	The <code>wishlist_items</code> row references a deleted product (cascade behavior depends on policy). To protect integrity: <code>wishlist_items.product_id</code> has <code>ON DELETE CASCADE</code> , so orphaned items are removed automatically before merge is attempted. Merge proceeds with remaining valid items.
EC-08	Concurrent merge requests for the same source from two tabs	The second request will find the source already deleted; returns a graceful "wishlist not found" error rather than a DB exception
EC-09	Wishlist name collision after merge	Names are not unique constraints; two wishlists can have the same name. No special handling needed.
EC-10	Very large wishlists (e.g., 1,000+ items)	The <code>INSERT ... ON CONFLICT</code> pattern handles large sets; pagination of item display is a separate UI concern. Transaction timeout is not a concern at realistic wishlist sizes.

10. Tradeoffs

10.1 Named Lists vs. Single Wishlist per User

- **Chosen:** Multiple named wishlists per user
- **Alternative:** Single wishlist per user (simpler, no merge needed)
- **Rationale:** The merge requirement is nonsensical with a single list. Multiple named lists is the only interpretation that makes the mandatory requirement meaningful.

10.2 Destructive Merge (Source Deleted) vs. Non-Destructive Copy

- **Chosen:** Destructive merge
- **Alternative:** Keep source intact after merge
- **Rationale:** Non-destructive merge results in confusion ("did I already merge this?") and violates the user's mental model of "merging." If non-destructive behavior is needed, it should be a separate "copy items" feature, not "merge."

10.3 Server-Side Transaction (Edge Function) vs. Client-Side Multi-Step

- **Chosen:** Server-side atomic operation
- **Alternative:** Client performs sequential API calls (get items → insert each → delete source)
- **Rationale:** Client-side multi-step is not atomic. A network failure between steps leaves the DB in an inconsistent state. Server-side is strictly safer, with no meaningful cost at this scale.

10.4 Deduplication by product_id Only vs. by product_id + variant

- **Chosen:** Deduplicate by product_id only (no product variants in V1)
- **Alternative:** Support variants (size, color) as separate items
- **Rationale:** Variants require a product variant sub-model that is out of scope. V1 products are flat. This is documented as a V2 concern.

10.5 Row Level Security vs. Application-Layer Authorization

- **Chosen:** Both (RLS as enforcement, application layer as UX gate)
- **Alternative:** Application-layer only
- **Rationale:** RLS is a defense-in-depth guarantee. Even if a bug in the application layer allows a malformed request, the DB refuses it. This is non-negotiable for multi-tenant data.

10.6 ON CONFLICT DO NOTHING vs. ON CONFLICT DO UPDATE

- **Chosen:** DO NOTHING (preserve original item's added_at)
- **Alternative:** DO UPDATE SET added_at = EXCLUDED.added_at (overwrite with source's timestamp)
- **Rationale:** The user added the item to the target first; that timestamp is chronologically correct. Overwriting it with the source timestamp would be confusing and provides no UX value.

11. Out of Scope (V1)

The following are explicitly excluded. They are acknowledged, documented, and deferred:

Feature	Deferral Reason
Shopping cart	Independent feature; not needed to demonstrate wishlist merge
Checkout / Payment	Requires payment gateway integration (Stripe, etc.)
Guest wishlist (localStorage → merge on login)	Complex session management; different architectural pattern
Wishlist sharing (public URL, social)	Requires visibility model + different RLS policies
Product variants (size, color)	Requires variant sub-model, complicates deduplication
Admin product management	Out of scope for 90-minute build; seed data suffices

Feature	Deferral Reason
Inventory / stock tracking	Requires real-time writes; not relevant to wishlist feature
Email notifications ("item back in stock")	Requires background jobs and transactional email
Multi-currency / tax calculation	External API dependency; not relevant to core scope
Wishlist analytics (most wishlisted products)	V2 analytics feature
Undo merge	Requires either soft delete or a separate history table

12. Open Questions (Require Product Decision Before V2)

1. **Should merge be reversible?** If yes, we need either soft deletes on wishlists or a `wishlist_merge_history` table.
2. **Should users be able to share a wishlist with another user?** If yes, the RLS model changes substantially (needs a `wishlist_collaborators` junction table).
3. **Should there be a maximum number of wishlists per user?** Currently unbounded. A limit (e.g., 20) might improve UX and prevent abuse.
4. **Should product variants be supported?** If yes, the unique constraint on (`wishlist_id`, `product_id`) must include a `variant_id`.
5. **What is the merge UX when the source has 0 new items to contribute?** Inform the user proactively ("All items already in target — merge will only delete the source list") to prevent surprises.

Specification complete. Implementation may begin once this document is reviewed and assumptions A1–A10 are confirmed.